Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Artificial Intelligence Practical

Practical – VII

Roll No. : T045	Name: Omkar Thukarul
Class : TYIT	Date/Time of Submission : 8/08/2026

Aim: Machine Learning

6). Implement Decision Tree regression on a regression dataset.

CODE:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
data =pd.read_csv("iris.csv")
X =data[['sepal_length']]
y=data['sepal_width']
X_train,X_test,y_train,y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1
)
model = DecisionTreeRegressor()
model.fit(X_train,y_train)
print("Training Accuracy:",model.score(X_train,y_train)*100)
print("Testing Accuracy:",model.score(X_test,y_test)*100)
new_data =pd.DataFrame([[11]],columns=['sepal_length'])
prediction = model.predict(new_data)
print("Predicted Score:",prediction[0])
```

OUTPUT:

```
Training Accuracy: 31.638143919277752
Testing Accuracy: 14.145810934928072
Predicted Score: 3.0666666666666664
```

7). Implement k-Nearest Neighbors (kNN) classifier.

CODE:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
# Load dataset
iris = load_iris()
X = iris.data
y = iris.target
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1
)
# Create kNN model
model = KNeighborsClassifier(n_neighbors=3)
# Train model
model.fit(X_train, y_train)
# Display accuracy
```

OUTPUT:

```
Training Accuracy: 95.23809523809523
Testing Accuracy: 97.77777777777777
Predicted Class: 0
```

```
print("Training Accuracy:", model.score(X_train, y_train)*100)
print("Testing Accuracy:", model.score(X_test, y_test)*100)
# Prediction
prediction = model.predict([X_test[0]])
print("Predicted Class:", prediction[0])
```

8).Implement multiclass classification using Logistic Regression or Decision Tree.

CODE:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
iris =load_iris()
X =iris.data
y=iris.target
X_train,X_test,y_train,y_test = train_test_split(
X,y,
test_size=0.3,
random_state=1
)
model=DecisionTreeClassifier()
model.fit(X_train,y_train)
print("Training Accuracy:",model.score(X_train,y_train)*100)
print("Testing Accuracy:",model.score(X_test,y_test)*100)
prediction = model.predict([X_test[0]])
print("predicted class:",prediction[0])
```

OUTPUT:

```
Training Accuracy: 100.0
Testing Accuracy: 95.55555555555556
Predicted Class: 0
```

Perform Logistic Regression, Decision Tree, and kNN on the same dataset. Compare the Results.

CODE:

```
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
# Load dataset and prepare binary target (1 = High price, 0 = Low price)
data = fetch_california_housing(as_frame=True)
X = data.data[['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms', 'Population', 'AveOccup', 'Latitude', 'Longitude']]
y = (data.target > data.target.median()).astype(int)
# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1)
# Scale features for Logistic Regression and kNN
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)
```

OUTPUT:

```
Logistic Regression - Train: 82.44739756367665 | Test: 82.91343669250647
Decision Tree       - Train: 100.0 | Test: 83.47868217054264
kNN Classifier      - Train: 90.8499446290144 | Test: 82.54198966408268
```

How Each Algorithm Works

Logistic Regression: Finds a single straight decision boundary (hyperplane) to separate the classes.

Decision Tree: Splits data into branches using a sequence of simple yes/no questions (e.g., Is income > $50k?).

kNN: Looks at the k nearest data points in space and predicts based on the majority vote of those neighbors.

Why the Results Differed

Linear vs. Non-Linear Patterns:

Housing prices depend on complex spatial relationships (like Latitude and Longitude).

Logistic Regression struggles because it can only fit a simple straight boundary, leading to moderate, balanced accuracy across train and test sets.

Overfitting (Decision Tree):

An unconstrained Decision Tree keeps splitting until it perfectly memorizes every training sample.

This produces 100% training accuracy, but because the rules become overly complex, test accuracy drops significantly.

Spatial Clustered Data (kNN):

kNN performs well on location data because houses physically close to each other tend to have similar values.

However, it relies heavily on calculated distances, making feature scaling (StandardScaler) strictly necessary so large features (like Population) don't dominate small features (like AveBedrms).